

Algoritma Pembangunan Pohon Hierarki

Penjelasan kode yang dibangun sendiri pada Sistem Ekstraksi Laporan Keuangan IDX — bagian rekonstruksi struktur hierarki akun.

1. Latar Masalah

Berkas PDF tidak menyimpan struktur tabel. Yang tersimpan hanyalah daftar **kata beserta koordinat**-nya, contoh: "Kas" di (x=94, y=358) . Tidak ada informasi mana baris induk dan mana baris anak.

Padahal akun laporan keuangan bersifat **hierarkis**. Misalnya *Giro pada bank lain* memiliki sub-akun *pihak ketiga, pihak berelasi, dan cadangan kerugian*. Tugas algoritma yang kami bangun adalah **merekonstruksi hierarki ini secara otomatis** dari data mentah, lalu menyimpannya dalam bentuk pohon (tree).

Sasaran: ubah data tidak terstruktur (kata + koordinat) menjadi data terstruktur berbentuk pohon dengan atribut yang kami tentukan sendiri (label, level, nilai, anak) sesuai *unified schema*.

2. Bentuk Keluaran: Unified Schema

Seluruh pendekatan menghasilkan struktur JSON yang sama (didefinisikan di `idx_fin_parser/unified.py`), sehingga dapat dibandingkan secara adil. Tiap simpul (node) pohon berbentuk:

```
{
  "level": 0, # kedalaman hierarki (0 = induk teratas)
  "label": "Giro pada bank lain",
  "label_en": "Current accounts with other banks",
  "row_type": "group", # group | item | label_only
  "values": { "2025-12-31": null, "2024-12-31": null },
  "children": [ ...node lain... ] # sub-akun (rekursif)
}
```

Atribut `level` dan `children` inilah yang dihasilkan oleh algoritma pembangunan pohon.

3. Algoritma Pembangunan Pohon (Dua Tahap)

Inti kontribusi kode kami terdiri atas dua tahap berurutan.

Tahap 1 — Menentukan Level dari Indentasi (`assign_levels`)

Lokasi: `idx_fin_parser/utils.py`. Ide dasarnya: **posisi horizontal (x) sebuah baris menunjukkan tingkat indentasinya**. Semakin menjorok ke kanan, semakin dalam levelnya.

1. Ambil `x0` (posisi kiri kata pertama) dari tiap baris yang memiliki angka
2. Kelompokkan (`cluster`) nilai-nilai `x0` dengan toleransi 8 poin
3. Cluster paling kiri -> level 0 (akun induk)
cluster berikutnya -> level 1 (sub-akun)
cluster berikutnya -> level 2, dan seterusnya
4. Tiap baris diberi level sesuai cluster `x0` terdekat

Contoh distribusi koordinat pada PDF BBNI:

Posisi <code>x0</code>	Level	Contoh baris
49	0	Aset / Kas / Giro pada bank lain
79	1	Giro pada bank lain pihak ketiga
94	2	Giro pihak ketiga (di bawah Simpanan nasabah)
109	3	Cadangan umum dan wajib

Tahap 2 — Menyusun Pohon dengan Stack (`_parse_structured_rows_to_tree`)

Lokasi: `idx_fin_parser/pdf_statements.py`. Setelah tiap baris punya level, pohon dibangun memakai struktur data **stack (tumpukan)** dengan aturan: simpul ber-level k menjadi anak dari simpul terakhir yang ber-level lebih kecil dari k .

```
stack = []
untuk setiap baris (urut atas ke bawah):

    # Jika baris adalah header seksi (Aset/Liabilitas/Ekuitas):
    jika baris == section header:
        section saat ini = baris
        stack = []          # reset, mulai seksi baru
        lanjut

    node = buat_node(baris)

    # Buang dari stack semua yang levelnya >= level baris ini
    selama stack tidak kosong DAN level(stack.puncak) >= level(baris):
        stack.pop()

    jika stack tidak kosong:
        node jadi ANAK dari stack.puncak
    selainnya:
        node jadi akar (langsung di bawah section)

    stack.push(node)
```

Ilustrasi Langkah demi Langkah

Memproses bagian Aset BBNI, mulai dari "Giro pada bank lain":

Baris masuk	Level	Aksi pada stack
Giro pada bank lain	0	stack kosong → jadi akar. <code>stack=[Giro]</code>
...pihak ketiga	1	<code>puncak(Giro,0) < 1</code> → anak Giro. <code>stack=[Giro, ketiga]</code>
...pihak berelasi	1	<code>puncak(ketiga,1) ≥ 1</code> → pop. <code>puncak(Giro,0) < 1</code> → anak Giro
Cadangan kerugian...	1	sama → pop berelasi, lalu anak Giro

Pohon yang terbentuk:

Giro pada bank lain		
└─ Giro pada bank lain pihak ketiga	25.858.094	22.031.212
└─ Giro pada bank lain pihak berelasi	79.999	42.974
└─ Cadangan kerugian penurunan nilai...	(7)	(13)

Kenapa pakai stack? Stack secara alami menjaga "jalur induk aktif". Ketika bertemu baris yang levelnya sama atau lebih dangkal, kita menutup cabang sebelumnya (pop) sampai menemukan calon induk yang tepat. Ini membuat hierarki sedalam apa pun terbentuk otomatis hanya dengan satu kali lintasan (O(n)).

4. Approach yang Memakai Algoritma Pembangunan Pohon Ini

Dari 8 pendekatan yang dibandingkan, tidak semuanya membangun pohon dengan cara yang sama. Terdapat **tiga sumber informasi level** yang berbeda, plus satu kelompok yang sengaja tidak membangun hierarki.

Approach	Sumber level	Fungsi pembangun pohon
native_pdf	Koordinat x0 (clustering)	<code>assign_levels</code> + <code>_parse_structured_rows_to_tree</code>
pymupdf_native	Koordinat x0 (clustering)	sama (pustaka PyMuPDF)
vlm_openai (GPT-4o)	Level dari output model	<code>flat_rows_to_tree</code> (stack, level dari model)
ocr_full (Tesseract)	Urutan teks + awalan label	<code>_parse_merged_lines_to_tree</code>
baseline_regex	— (tidak ada)	flat, semua level 0
pdfplumber_tables	— (tidak ada)	flat, semua level 0
camelot_stream	— (tidak ada)	flat, semua level 0

Penjelasan tiap kelompok

(a) Berbasis koordinat — native_pdf & pymupdf_native. Memakai algoritma dua tahap di Bab 3 (clustering x0 + stack). Paling akurat karena setia pada tata letak fisik dokumen. Keduanya memakai

`_parse_structured_rows_to_tree` yang sama; bedanya hanya pustaka pengambil koordinat (pdfplumber vs PyMuPDF).

(b) Berbasis output model — vlm_openai. Model VLM (GPT-4o) "membaca" indentasi langsung dari gambar dan mengeluarkan angka `level` di JSON-nya. Kode kami tinggal membangun pohon dengan prinsip stack yang sama, namun level tidak dihitung dari koordinat melainkan diberikan oleh model. Fungsi: `flat_rows_to_tree` di `unified.py`.

(c) Berbasis teks — ocr_full. Karena OCR (Tesseract) kehilangan koordinat, hierarki ditebak dari urutan baris dan pencocokan awalan label (jika label suatu baris diawali teks label baris di atasnya, dianggap anak). Jauh lebih lemah; ini sebab metrik hierarki OCR rendah. Fungsi: `_parse_merged_lines_to_tree`.

(d) Tanpa hierarki — baseline_regex, pdfplumber_tables, camelot_stream. Sengaja tidak membangun pohon: semua baris diletakkan sejajar pada level 0. Berfungsi sebagai pembanding "tanpa usaha khusus" untuk menunjukkan seberapa besar kontribusi algoritma pembangunan pohon.

Kesimpulan untuk laporan: algoritma pembangunan pohon (clustering indentasi + penyusunan stack) adalah komponen orisinal sistem. Hanya dua approach koordinat (`native_pdf`, `pymupdf_native`) yang memakainya secara penuh; VLM memakai prinsip stack tetapi levelnya dari model; OCR memakai heuristik teks; sisanya tidak membangun hierarki sama sekali.

5. Ringkasan

1. **Unified schema** menetapkan bentuk akhir: pohon node dengan atribut level, label, nilai, dan anak.
2. **Tahap 1 (assign_levels):** indentasi fisik (x0) → angka level melalui clustering.
3. **Tahap 2 (stack-based attach):** menyusun node menjadi pohon induk-anak dalam satu lintasan.
4. Algoritma ini dipakai penuh oleh **native_pdf** dan **pymupdf_native**; **VLM** memakai prinsip stack dengan level dari model; **OCR** memakai heuristik teks; **baseline/camelot/pdfplumber** tidak membangun hierarki.

Dokumen penjelasan kode — Sistem Otomasi Ekstraksi Laporan Keuangan IDX. Berkas terkait: `idx_fin_parser/utils.py`, `idx_fin_parser/pdf_statements.py`, `idx_fin_parser/unified.py`.